

1. Descripción Arquitectónica y Esquema de Capas

El sistema se ha diseñado bajo un patrón de **Arquitectura de N-Capas**, fundamentado en los principios de **Clean Architecture** para garantizar el desacoplamiento y la mantenibilidad. La solución se segmenta en:

- **Capa de Presentación (Frontend):** Desarrollada en Angular, encargada exclusivamente de la captura de datos mínimos y la visualización de estados.
- **Capa de Orquestación y Aplicación (Backend - NestJS):** Centraliza la lógica de negocio, validaciones de seguridad y coordinación de servicios externos.
- **Capa de Dominio y Persistencia:** Utiliza MongoDB para el almacenamiento persistente y Redis como capa de aceleración y caché transaccional.

Seguridad y Privacidad: La protección de la información se garantiza mediante la implementación de **Guards de Autenticación** a nivel de controlador. El esquema asegura que la lógica de negocio sea inaccesible sin un **JSON Web Token (JWT)** válido, cifrado y firmado, mitigando riesgos de acceso no autorizado a los endpoints de procesamiento.

2. Estrategia de Recuperación y Enriquecimiento de Datos

Para optimizar el rendimiento y cumplir con la restricción de minimizar consultas recurrentes a la base de datos persistente, se implementa una estrategia de **"Data Hydration"** en dos fases:

1. **Fase de Autenticación:** Durante el proceso de Login, el sistema recupera el perfil transaccional completo del usuario desde la base de datos principal y lo almacena en una **caché de alta velocidad (Redis)** con un tiempo de vida (TTL) definido.
2. **Fase de Procesamiento:** Al recibir una solicitud de crédito, el backend no re-consulta la base de datos; en su lugar, recupera el contexto del solicitante directamente desde Redis. Este enfoque permite que el motor de IA disponga de información enriquecida (ingresos, historial, perfil de riesgo) de forma instantánea en múltiples etapas del flujo sin degradar el rendimiento del sistema.

3. Desacoplamiento de Seguridad y Lógica de Negocio

En el diseño propuesto, los servicios de procesamiento (IA/Cálculo de Score) son **agnósticos a la identidad**. La responsabilidad de la seguridad es asumida por una capa superior de **Middleware y Guards**. Esta separación de responsabilidades asegura que el motor de procesamiento reciba únicamente los datos necesarios para su ejecución, habiendo sido previamente saneados y autorizados por la capa de aplicación. Esto permite, en un entorno productivo, extraer el motor de IA a un microservicio independiente sin necesidad de replicar la lógica de autenticación.

4. Enfoque Arquitectónico Multi-Interfaz (Web y Móvil)

Para soportar interfaces Web y Móvil con experiencias nativas diferenciadas, se recomienda un enfoque de **Backend for Frontends (BFF)**. Bajo esta arquitectura:

- Se mantiene un núcleo de API robusto y común.
- Se implementa una capa de presentación adaptativa que consume el mismo contrato de servicios, pero utiliza librerías de componentes específicas de plataforma (Material Design para entorno Web y elementos nativos mediante Capacitor/Ionic para entorno móvil). Esto garantiza que la lógica de negocio sea única, mientras que el *Look & Feel* y el manejo de gestos se ajusten a los estándares de cada dispositivo.

5. Manejo de Sesión y Protección de Información Sensible

El manejo de sesión se fundamenta en **JWT**, donde el token actúa estrictamente como un identificador de identidad (applicantId). Para proteger la información sensible:

- **Aislamiento de Datos:** El Frontend nunca recibe información confidencial que no sea estrictamente necesaria para la vista actual.
- **Optimización de Consultas:** Se evita la consulta redundante a la base de datos mediante la persistencia temporal en Redis. Esto resuelve el requerimiento de procesar solicitudes que dependen de datos históricos del usuario sin exponer dichos datos en el tráfico de red hacia el cliente ni saturar el motor de base de datos relacional/documental.

6. Integración con Single Sign On (SSO) y Manejo de Sesión

A nivel de diseño arquitectónico para un entorno productivo, la integración con SSO se proyecta mediante el protocolo OpenID Connect (OIDC), delegando la gestión de identidad a un servidor federado (ej. Keycloak o Auth0). Sin embargo, para el alcance de esta prueba técnica y con el objetivo de mantener el proyecto autocontenido, se optó por **simular este comportamiento mediante un microservicio dedicado (auth-sso)**.

Este microservicio independiente actúa como nuestro propio proveedor de identidad: recibe las credenciales, las valida y emite un token JWT (JSON Web Token) firmado. De esta manera, se cumple y demuestra el principio de desacoplamiento del SSO: el API Gateway y el Backend de Créditos nunca interactúan con contraseñas ni lógica de usuarios, limitándose a confiar y verificar la validez del token delegado por el servicio de autenticación.

7. Escalabilidad y Modelo de Despliegue

Para asegurar la escalabilidad ante volúmenes transaccionales masivos, se proponen las siguientes decisiones técnicas:

- **Modelo de Despliegue:** Contenerización mediante **Docker**, orquestada en un clúster de **Kubernetes (K8s)**. Esto permite el escalado horizontal automático de los pods de procesamiento según la demanda de CPU/RAM.
- **Escalabilidad de Datos:** Implementación de **Sharding** en MongoDB y replicación de lectura en Redis para distribuir la carga de consultas.
- **Límites de Responsabilidad:** Se establece un límite claro donde el Backend actúa como orquestador de flujos de trabajo (Workflow Orchestrator), mientras que el procesamiento intensivo de IA se delega a trabajadores asíncronos mediante **Colas de Mensajería (Message Brokers)**, evitando bloqueos en la interfaz de usuario y permitiendo el procesamiento concurrente de miles de solicitudes.